Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 3 - Docker storage

Question 3: Docker Storage - Demonstrate loss of container-layer data after container deletion, and permanent storage using a Docker named volume and bind mount.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64), Docker Engine 29.9.3, using `ubuntu:24.04` containers. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Demo 1 - container writable layer is lost

```
$ docker run -d --name s-layer ubuntu:24.04 sleep infinity
$ docker exec s-layer sh -c 'echo samir-layer-note > /tmp/note.txt; cat /tmp/note.txt'
samir-layer-note
$ docker rm -f s-layer
$ docker run -d --name s-layer ubuntu:24.04 sleep infinity
$ docker exec s-layer cat /tmp/note.txt
cat: /tmp/note.txt: No such file or directory
```

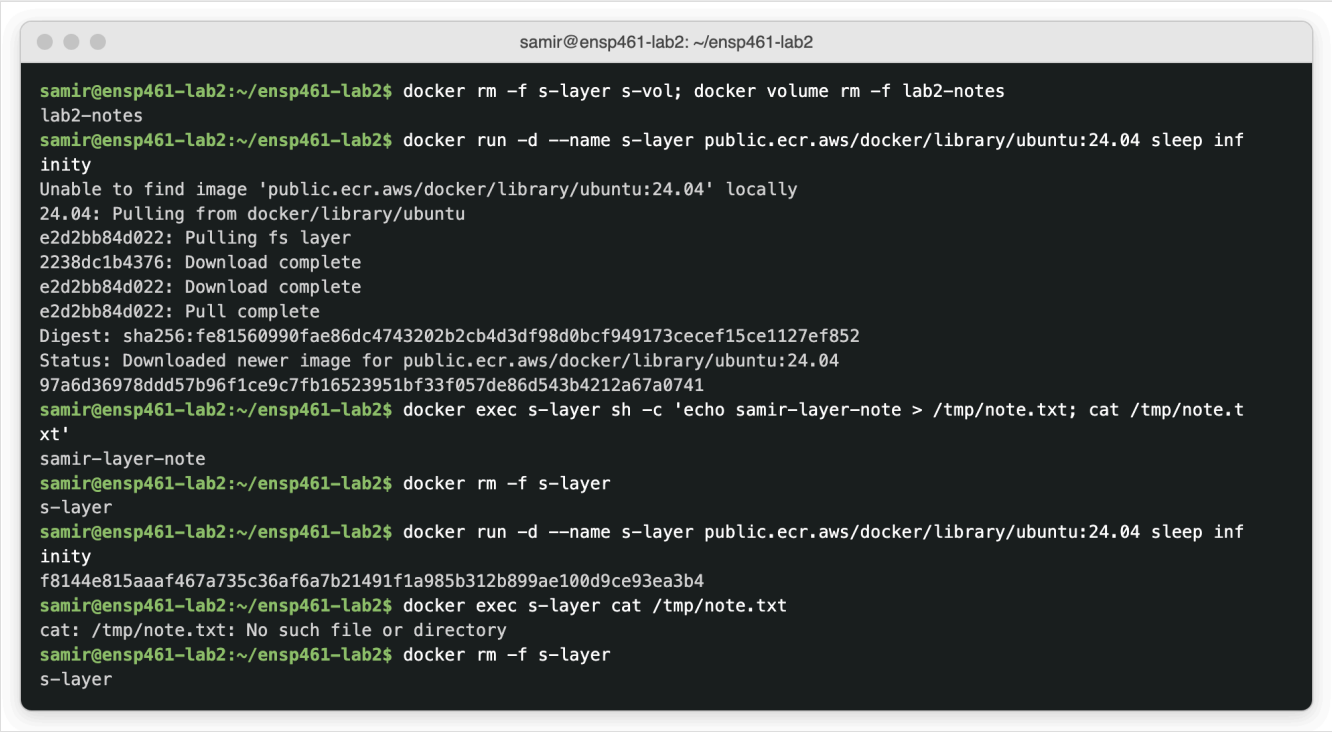


Fig: Writable-layer data disappears with the container

Every container gets a fresh thin writable layer on top of the read-only image layers. Removing the container deletes that layer, so a new container from the same image starts clean.

Demo 2 - named volume survives

```
$ docker volume create lab2-notes
$ docker run -d --name s-vol -v lab2-notes:/data ubuntu:24.04 sleep infinity
$ docker exec s-vol sh -c 'echo samir-volume-note > /data/note.txt'
$ docker rm -f s-vol
$ docker run --rm -v lab2-notes:/data ubuntu:24.04 cat /data/note.txt
samir-volume-note
$ docker volume inspect lab2-notes --format '{{.Name}} {{.Driver}} {{.Mountpoint}}'
lab2-notes local /var/lib/docker/volumes/lab2-notes/_data
```

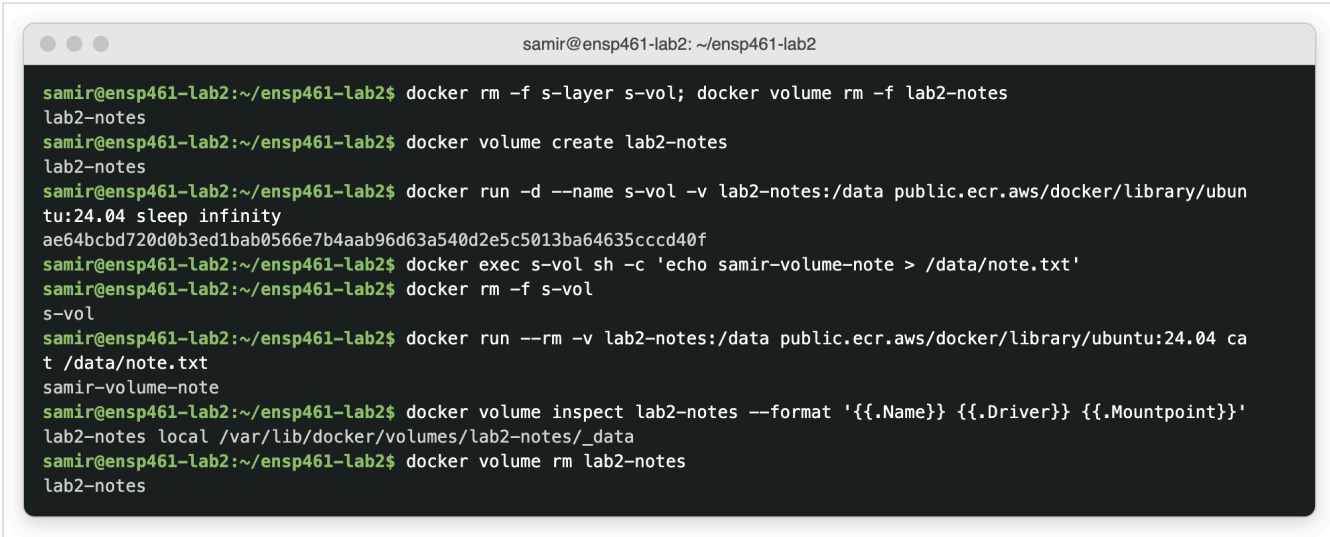


Fig: Named volume keeps data across containers

The volume is managed by Docker (`local` driver, stored under `/var/lib/docker/volumes`) and has its own lifecycle. It outlives any container and is only removed by `docker volume rm` (or `down -v` in Compose, as in Q2).

Demo 3 - bind mount shares a host directory

```
$ echo from-host > '/home/samir/ensp461-bind/shared.txt'
$ docker run --rm -v '/home/samir/ensp461-bind':/data ubuntu:24.04 cat /data/shared.txt
from-host
$ docker run --rm -v '/home/samir/ensp461-bind':/data ubuntu:24.04 sh -c 'echo from-container >>
/data/shared.txt'
$ cat '/home/samir/ensp461-bind/shared.txt'
from-host
from-container
```

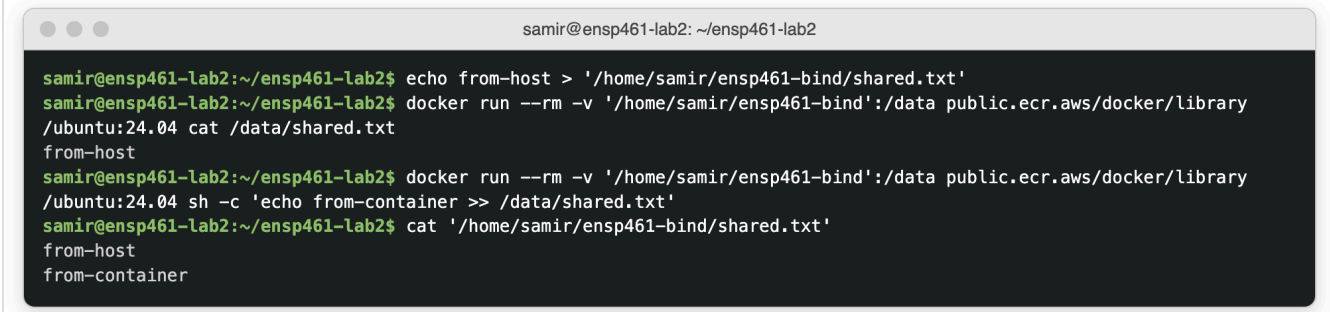


Fig: Bind mount: host and container edit the same file

A bind mount maps an exact host path into the container, so writes are visible on both sides immediately.

Comparison

	Container layer	Named volume	Bind mount
Survives `docker rm`	No	Yes	Yes
Location	inside the container's layer	`/var/lib/docker/volumes/<name>`	any host path you choose
Managed by	Docker, tied to container	Docker, independent	the host user
Good for	scratch files	databases, app state	source code, config, logs

Problem met and fix

A first attempt ran the bind-mount demo from the macOS Mac's `Desktop` folder. The container's write failed with `Operation not permitted`, because macOS privacy controls restrict access to `Desktop`. The lab was moved into a dedicated Ubuntu 24.04 OrbStack machine, and the bind directory into its Linux home (`/home/samir/ensp461-bind`), where it worked as shown.